

Zugang einrichten. Gemeinsam arbeiten.

Zum gemeinsamen Durchgehen im Call: links steht, was Jonas macht, rechts, was der Kollege macht. Beide Spuren laufen parallel - abhaken und weiter.

Grundregel: Kein geteilter Ordner, kein geteiltes Passwort. Jeder arbeitet mit dem eigenen GitHub-Konto am selben Repository. Änderungen laufen immer über einen Branch und einen Pull Request.

Schritt	Wer	Dauer	Ergebnis
1 Daten sammeln	Kollege	2 Min	GitHub-Handle, MFA aktiv, Rolle geklärt
2 main schützen	Jonas	3 Min	Kein direkter Push auf die Live-Seite möglich
3 Einladung senden	Jonas	1 Min	Kollege erhält Write-Zugang
4 Einladung annehmen	Kollege	1 Min	Zugriff aktiv
5 Projekt einrichten	Kollege	10 Min	Repo lokal, npm ci gelaufen, Vorschau läuft
6 Teständerung	Beide	15 Min	Ein Pull Request einmal komplett durchgespielt

Adressbezug: „wirtschaft“ cafe restaurant bar, Bahnhofstraße 24, 6850 Dornbirn · Repository:
github.com/jonasgamper-create/wirtschaft-dornbirn-test

Vor dem Call: drei Angaben

Diese drei Dinge braucht Jonas vom Kollegen, bevor die Einladung rausgeht. Am besten schon vorab per Nachricht schicken, dann dauert der Call nur halb so lang.

Was	Wo findet er das	Warum
GitHub-Benutzername	Oben rechts im Profilmenu oder in der Profil-URL github.com/	Die Einladung geht an den Handle, nicht an die E-Mail. Ein Tippfehler lädt eine fremde Person ein.
Bestätigung: MFA ist aktiv	github.com/settings/security – Zwei-Faktor oder Passkey einschalten	Ohne zweiten Faktor kein Zugang. Das Repo enthält die Quelle der Live-Seite.
Gewünschte Rolle	Read = nur mitlesen. Write = Branches und Pull Requests anlegen.	Empfehlung: Write. Damit kann er arbeiten, aber nichts löschen und keine Einstellungen ändern.

Niemals per Nachricht schicken: Passwörter, Wiederherstellungscodes, API-Schlüssel, Zugangsdaten von Reservierungs- oder Ticketanbietern. Der Zugang läuft ausschließlich über die GitHub-Einladung.

Der Call: Schritt für Schritt

1 Daten prüfen

JONAS	KOLLEGE
Handle laut vorlesen und gemeinsam gegen das Profil prüfen. Bei Namensgleichheit: Profilbild und Repos vergleichen.	Profil offen halten, damit der Handle zweifelsfrei stimmt. Zwei-Faktor gemeinsam kurz zeigen.

Genau hier passieren die Fehler: ähnliche Handles sind häufig. Lieber 30 Sekunden länger schauen.

2 main schützen – zuerst, nicht später

JONAS	KOLLEGE
Settings → Branches → Add branch ruleset. Branch main wählen, „Require a pull request before merging“ mit 1 Review aktivieren, speichern.	Zuschauen. Der Schutz gilt ab sofort auch für Jonas selbst – das ist gewollt.

Wichtiger als die Einladung: Ohne diesen Schutz kann ein versehentlicher Push die Live-Seite verändern.

3 Einladung senden

JONAS	KOLLEGE
Settings → Collaborators and teams → Add people. Handle eingeben, Rolle Write wählen, bestätigen.	Noch nichts tun. Warten, bis die Benachrichtigung ankommt.

4 Einladung annehmen

JONAS	KOLLEGE
Kurz bestätigen lassen, dass der Zugriff sichtbar ist.	github.com/notifications öffnen oder den Link aus der E-Mail nutzen → Accept invitation . Danach das Repository einmal öffnen.

Projekt einrichten und ein Pull Request üben

5

Projekt lokal einrichten

JONAS	KOLLEGE
Beim ersten Durchlauf mitschauen. Node 20 oder neuer wird gebraucht.	Terminal öffnen und die drei Befehle der Reihe nach ausführen (siehe unten). Danach die Vorschau im Browser öffnen.

```
git clone https://github.com/jonasmamper-create/wirtschaft-dornbirn-test.git
cd wirtschaft-dornbirn-test
npm ci
npx http-server site -p 8123
```

Die Vorschau läuft danach auf <http://localhost:8123>. Das ist die lokale Arbeitsansicht, nicht die Live-Seite.

6

Eine kleine Änderung komplett durchspielen

JONAS	KOLLEGE
Den Pull Request prüfen: Diff lesen, Mobile bei 390 px ansehen, Buchungswege klicken, CI-Ergebnis kontrollieren. Erst dann mergen.	Branch anlegen, eine Kleinigkeit ändern, npm run ci laufen lassen, committen, pushen und den Pull Request öffnen.

Einmal gemeinsam durchspielen lohnt sich – danach sitzt der Ablauf.

```
git checkout -b feature/kurzer-name
# ... Änderung machen ...
npm run ci
git add -A && git commit -m "kurze beschreibung"
git push -u origin feature/kurzer-name
```

npm run ci muss grün sein, bevor ein Pull Request gemergt wird. Die Prüfung deckt Build, Eventdaten, Texte, Public-Build, Datenschutz, Interaktionen und die Mittagskarte ab.

Spielregeln – nicht verhandelbar

Diese Punkte gelten für beide gleichermaßen. Sie stehen so auch in CLAUDE.md und SECURITY.md im Repository.

Thema	Regel	Warum
Branches	Nie direkt auf main. Jede Änderung als Branch mit Präfix feature/ und Pull Request.	main veröffentlicht die Testseite.
Eventdaten	Nur aus site/data/events.json, vorher gegen die offizielle Eventseite geprüft.	Erfundene Termine, Preise oder ein falscher Status kosten Vertrauen und Buchungen.
Mittagskarte	Nur aus site/data/lunch-menu.json. Keine Gerichte oder Preise erfinden.	Eine falsche Karte ist für den Gast ein gebrochenes Versprechen.
Buchung	Reservierung und Tickets bleiben Weiterleitungen zu den offiziellen Anbietern.	Die Website verarbeitet keine Zahlungs- oder Gästedaten.
Daten	Keine echten Gäste-, Zahlungs-, Login- oder API-Daten in Code, Screenshots, Issues oder KI-Prompts.	DSGVO und Sicherheit. Einmal veröffentlicht ist nicht zurückzuholen.
Tracker	Keine Analyse- oder Marketing-Skripte ohne geprüfte Einwilligung.	Vorab-Einwilligung ist Pflicht, nicht Kür.
Bilder	Große Rohbilder und Videos in den geschützten Drive-Ordner, nicht ins Repository.	Das Repository bleibt schlank und versionierbar.

Ablage und Links

Ablage außerhalb des Codes

Website und Code leben im Repository. Alles andere gehört in den Drive-Ordner. Kampagnen- und Projektordner immer mit Datum benennen, zum Beispiel **2026-08-09_kampagne-herbst**.

Ordner	Inhalt
01_Briefing-und-Freigaben	Schriftliche Freigaben von Wolfgang, Briefings, Protokolle
02_Originalbilder-und-Videos	Rohmaterial mit Nutzungsrechten
03_Eventdaten-und-Texte	Termine, Texte, Menü-Entwürfe vor der Freigabe
04_Rechtliches-DSGVO-Impressum	Verträge, AVV, Rechtstexte
05_Social-Export	Fertige Posts und Stories
06_Archiv	Abgeschlossenes
07_Performance-Marketing	Kampagnen, Budgets, Auswertungen – je Kampagne ein Datumsordner

Wichtige Links

Zweck	Link
Repository und Aufgaben	https://github.com/jonasgamper-create/wirtschaft-dornbirn-test
Testseite ansehen	https://jonasgamper-create.github.io/wirtschaft-dornbirn-test/
Offizielle Eventquelle	https://wirtschaft-dornbirn.at/event/
Tischreservierung (offiziell)	https://tischreservierung.wirtschaft-dornbirn.at/
Ausführliche Anleitung im Repo	docs/onboarding-kollege.md
Arbeitsregeln im Repo	CLAUDE.md und SECURITY.md

Die Testseite steht bewusst auf noindex und ist kein Produktivsystem: keine echten Reservierungen, keine Zahlungen, keine Gästedaten. Sie zeigt den Stand des main-Branches – lokale Arbeitsstände sind dort noch nicht sichtbar.